

Sistema de Gestão Comercial para Pequenos Negócios (SGC) -

THE Way SUPPLEMENTS

Alunos: João Francisco Torres e Marcus Vinícius Rodrigues Bacelar;

Disciplina: Desenvolvimento Web;

Professor: Felipe Pires Ferreira.

Alunos:

João Francisco Torres e Marcus
Vinícius R. Bacelar;

Disciplina:

Desenvolvimento Web;

Professor:

Felippe Pires Ferreira.



O SISTEMA

O SISTEMA:

TheWay é um sistema de gestão comercial para lojas de suplementos alimentares. Ele centraliza o cadastro de clientes, produtos, estoque e vendas em uma única plataforma.

O SISTEMA:



CONFIGURAÇÃO

settings.py:

Este arquivo registra tudo que o Django precisa saber sobre o projeto:
apps instalados, banco de dados, JWT e permissões padrão da API.

Apps instalados:

config/settings.py

```
INSTALLED_APPS = [ # Apps nativos do Django
    'django.contrib.admin', 'django.contrib.auth', ... #
    Bibliotecas instaladas via pip 'rest_framework', # Django REST
    Framework (API REST) 'rest_framework_simplejwt', # Autenticação
    JWT 'corsheaders', # Permite o frontend chamar a API # Nossos
    apps criados no projeto 'produtos', 'clientes', 'vendas',
    'suplementos', ]
```

O que cada biblioteca faz?

- **rest_framework** — transforma o Django em uma API REST, com suporte a JSON, ViewSets e Routers
- **rest_framework_simplejwt** — adiciona as rotas de login que geram o token JWT
- **corsheaders** — sem isso, o navegador bloqueia as chamadas do frontend para a API (política de segurança chamada CORS)

Configuração do JWT e Permissões:

config/settings.py

```
REST_FRAMEWORK = { # Define que toda autenticação usa JWT
    'DEFAULT_AUTHENTICATION_CLASSES': [
        'rest_framework_simplejwt.authentication.JWTAuthentication', ],
    # Define que toda rota exige usuário logado
    'DEFAULT_PERMISSION_CLASSES': [
        'rest_framework.permissions.IsAuthenticated', ], # Handler
    # customizado para nossos erros (ex: estoque)
    'EXCEPTION_HANDLER': 'core.handlers.custom_exception_handler',
} SIMPLE_JWT = { 'ACCESS_TOKEN_LIFETIME': timedelta(hours=8), #
    # token válido por 8h 'REFRESH_TOKEN_LIFETIME':
    # timedelta(days=1), # refresh válido por 1 dia }
```

Por que isso é importante?

As duas configurações juntas fazem com que **qualquer endpoint da API** automaticamente exija um token válido — sem precisar repetir isso em cada view. É uma configuração global.

ROTAS

`urls.py`:

Este arquivo conecta cada URL a uma view (função ou classe que processa a requisição).

urls.py:

config/urls.py

```
urlpatterns = [ # Painel administrativo do Django
path('admin/', admin.site.urls), # Rotas de autenticação JWT
(fornecidas pela biblioteca) path('api/token/',
TokenObtainPairView.as_view()), # login
path('api/token/refresh/', TokenRefreshView.as_view()), #
renovar token # Nossas rotas de API REST (geradas
automaticamente pelo Router) path('api/',
include('produtos.urls')), # /api/produtos/ path('api/',
include('clientes.urls')), # /api/clientes/ path('api/',
include('vendas.urls')), # /api/vendas/ # Interface web
(frontend e recomendação) path('',
include('suplementos.urls')), ]
```

Como o Router gera as rotas automaticamente?

Quando usamos **DefaultRouter** nos apps, ele cria automaticamente as 5 rotas padrão de um CRUD: listar (GET), criar (POST), detalhar (GET `/{{id}}/`), atualizar (PUT `/{{id}}/`), excluir (DELETE `/{{id}}/`). Não precisamos escrever cada rota na mão.

BANCO DE DADOS

models.py:

Os models definem a estrutura do banco de dados usando classes Python.

O Django converte isso em SQL automaticamente quando rodamos migrate .

Model de Venda (relacionamentos):

vendas/models.py

```
class Venda(models.Model): STATUS = [ ('ABERTA', 'ABERTA'),  
    ('CONCLUIDA', 'CONCLUIDA'), ('CANCELADA', 'CANCELADA'), ]  
data_hora = models.DateTimeField(auto_now_add=True) #  
preenchido automaticamente status =  
models.CharField(choices=STATUS, default='CONCLUIDA') total =  
models.DecimalField(max_digits=10, decimal_places=2, default=0)  
cliente = models.ForeignKey(Cliente, on_delete=models.PROTECT)  
# chave estrangeira class ItemVenda(models.Model): venda =  
models.ForeignKey(Venda, on_delete=models.CASCADE,  
    related_name='itens') produto = models.ForeignKey(Produto,  
    on_delete=models.PROTECT) quantidade = models.IntegerField()  
preco_unitario = models.DecimalField(max_digits=10,  
    decimal_places=2) subtotal = models.DecimalField(max_digits=10,  
    decimal_places=2)
```

Pontos importantes deste model

- **ForeignKey** — cria um relacionamento entre tabelas. Uma Venda pertence a um Cliente; um ItemVenda pertence a uma Venda e a um Produto
- **on_delete=PROTECT** — impede excluir um cliente que tem vendas registradas
- **on_delete=CASCADE** — se a venda for excluída, os itens dela são excluídos junto
- **related_name='itens'** — permite acessar os itens de uma venda com `venda.itens.all()`

Model de Recomendação:

suplementos/models.py

```
class ConsultaSuplemento(models.Model): nome =  
models.CharField(max_length=100) peso = models.FloatField()  
altura = models.FloatField() recomendacao =  
models.TextField(blank=True) def save(self, *args, **kwargs): #  
Calcula o IMC antes de salvar no banco imc = self.peso /  
(self.altura ** 2) if imc < 18.5: self.recomendacao = "Foco em  
Hipercalóricos e Proteínas." elif 18.5 <= imc < 25:  
self.recomendacao = "Foco em Creatina e Whey Protein." else:  
self.recomendacao = "Foco em Termogênicos e controle de dieta."  
super().save(*args, **kwargs) # chama o save original do Django
```

Por que sobrescrever o save()?

Sobrescrevemos o método `save()` para executar a lógica do IMC automaticamente toda vez que uma consulta é salva — sem precisar lembrar de chamar isso manualmente em outro lugar. O `super().save()` no final garante que o Django ainda execute o comportamento padrão de salvar no banco.

SERIALIZERS

serializers.py:

Os serializers são responsáveis por duas coisas: converter dados do banco em JSON (para a resposta) e validar os dados recebidos do frontend (antes de salvar).

Validação de CPF com dígitos verificadores:

clientes/serializers.py

```
def validate_cpf(self, value): # Remove tudo que não for número
    (pontos, traço) cpf = re.sub(r'\D', '', value) # Verifica se
    tem 11 dígitos if len(cpf) != 11: raise
    serializers.ValidationError("CPF deve conter 11 dígitos.") #
    Rejeita CPFs como 111.111.111-11 (todos iguais) if cpf ==
    cpf[0] * 11: raise serializers.ValidationError("CPF inválido.")
    # Valida os 2 dígitos verificadores (algoritmo da Receita
    Federal) for i in range(9, 11): soma = sum(int(cpf[j]) * (i + 1
    - j) for j in range(i)) digito = (soma * 10 % 11) % 10 if
    digito != int(cpf[i]): raise serializers.ValidationError("CPF
    inválido.") return value
```

O que são dígitos verificadores?

Os dois últimos dígitos do CPF são calculados matematicamente a partir dos 9 primeiros. Esse algoritmo da Receita Federal garante que um CPF digitado aleatoriamente (como 123.456.789-00) seja rejeitado. Não é só checar se tem 11 números — é uma validação matemática real.

Lógica de criação de venda com controle de estoque:

vendas/serializers.py

```
def create(self, validated_data): itens_data =
validated_data.pop('itens') # separa os itens venda =
Venda.objects.create(**validated_data) # cria a venda total = 0
for item in itens_data: produto =
Produto.objects.get(id=item['produto'].id) quantidade =
item['quantidade'] # Verifica se há estoque suficiente if
produto.qtd_estoque < quantidade: venda.delete() # desfaz a
venda raise EstoqueInsuficienteException(produto.nome) subtotal
= produto.preco * quantidade ItemVenda.objects.create(
venda=venda, produto=produto, quantidade=quantidade,
preco_unitario=produto.preco, # captura o preço atual
subtotal=subtotal ) produto.qtd_estoque -= quantidade #
desconta o estoque produto.save() total += subtotal venda.total
= total venda.save() return venda
```

Regras de negócio implementadas aqui

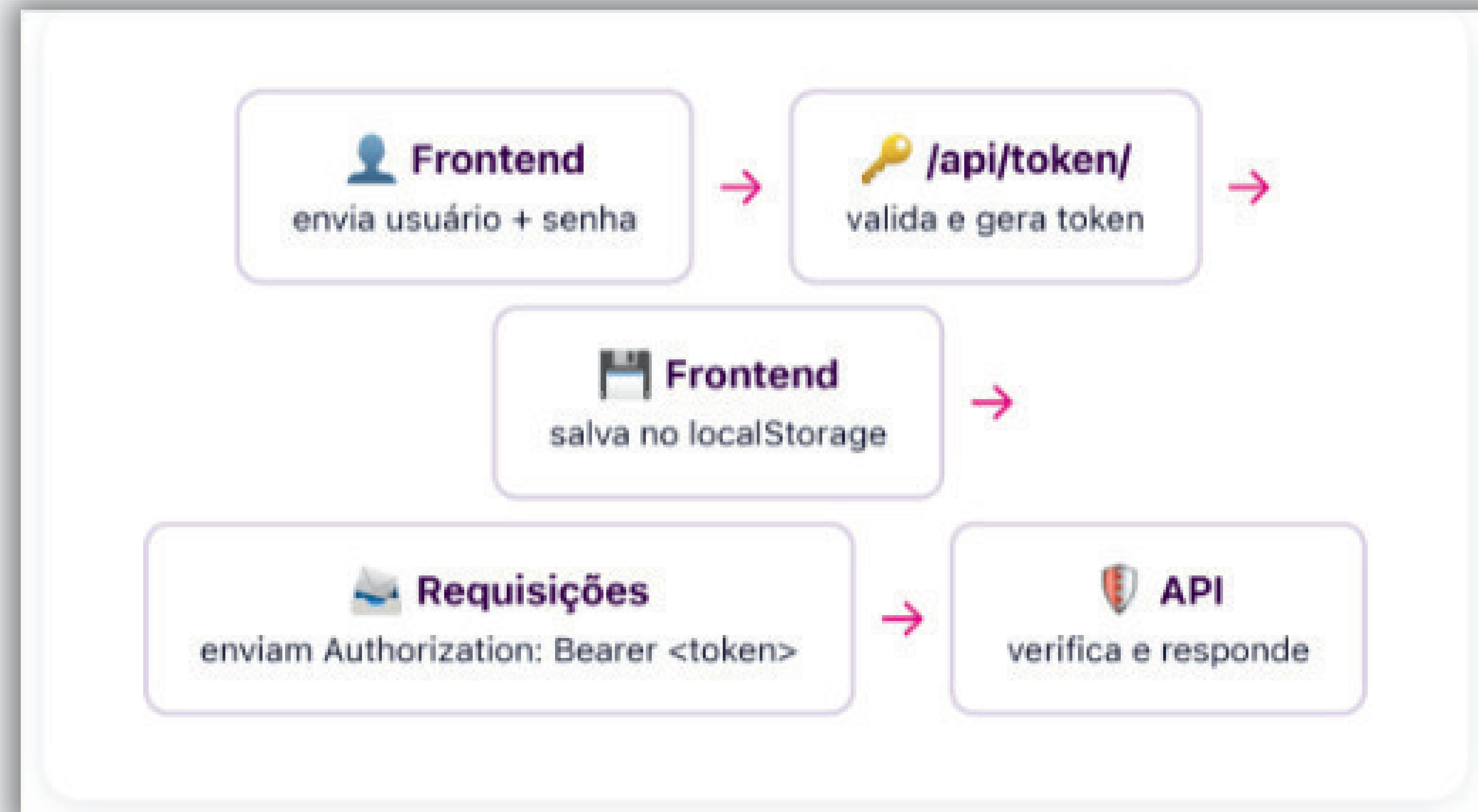
- **Verificação de estoque** — se qualquer item não tiver estoque suficiente, a venda inteira é cancelada (`venda.delete()`)
- **Captura do preço** — o `preco_unitario` é salvo no momento da venda, então se o preço do produto mudar depois, o histórico fica correto
- **Desconto automático** — o estoque é decrementado produto a produto no mesmo loop
- **Total calculado** — o campo `total` da venda é sempre calculado pelo servidor, nunca enviado pelo frontend

SEGURANÇA

Como o JWT funciona na prática?

JWT (JSON Web Token) é um token criptografado que o servidor emite após o login. Ele prova que o usuário está autenticado sem precisar checar o banco a cada requisição.

Como o JWT funciona na prática?



Como o frontend envia o token?

frontend.html (JavaScript)

// Função auxiliar que já injeta o token em toda requisição

```
async function api(method, url, body) { const opts = { method,
headers: { 'Content-Type': 'application/json', 'Authorization':
'Bearer ' + token // <-- token JWT aqui } }; if (body)
opts.body = JSON.stringify(body); const res = await fetch(url,
opts); // Se o token expirou, faz logout automático if
(res.status === 401) { doLogout(); return null; } return res; }
```

Por que centralizar assim?

Em vez de repetir o cabeçalho **Authorization** em cada chamada, criamos uma função **api()** que já faz isso automaticamente. Todas as chamadas do sistema passam por ela — se o token expirar, o logout é automático.

EXECUÇÃO

Como rodar o projeto:

1

Instalar as dependências

Bibliotecas que não vieram com o Django e foram adicionadas na Entrega 3:

```
pip install djangorestframework-simplejwt  
django-cors-headers
```

2

Entrar na pasta correta

O `manage.py` fica dentro de `TheWay_Projeto`, não na raiz do ZIP:

```
cd "...TheWay-entrega3\TheWay_Projeto"
```

3

Aplicar as migrações

Cria todas as tabelas no banco SQLite com base nos models:

```
python manage.py migrate
```

4

Criar o superusuário

Necessário para fazer login na interface. Sugestão: usuário

`admin`, senha `admin123` :

```
python manage.py createsuperuser
```


Como rodar o projeto:

5

Iniciar o servidor

Sobe o servidor de desenvolvimento na porta 8000:

```
python manage.py runserver
```

6

Acessar no navegador

Interface web completa (login) e recomendação de suplementos (público):

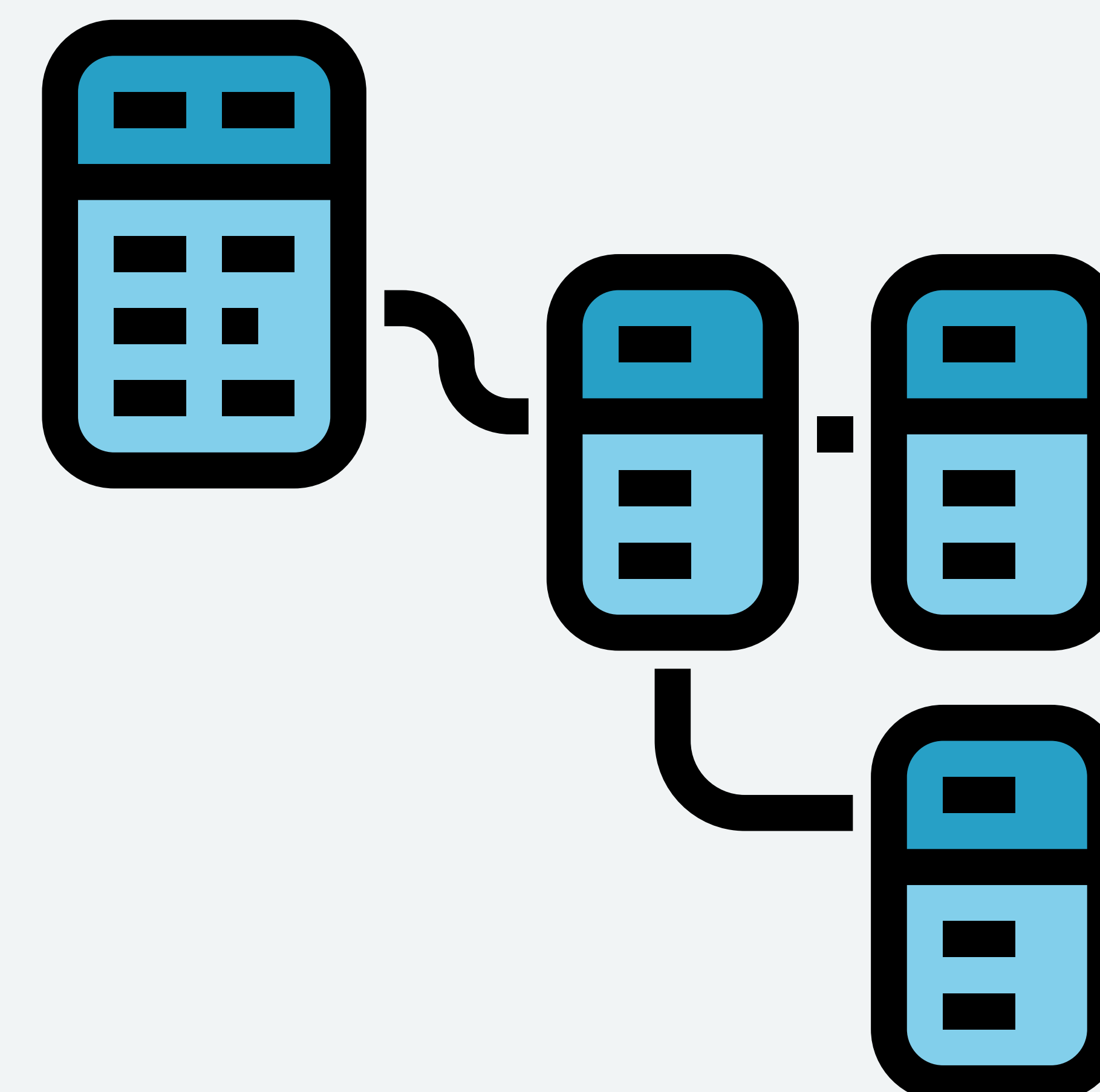
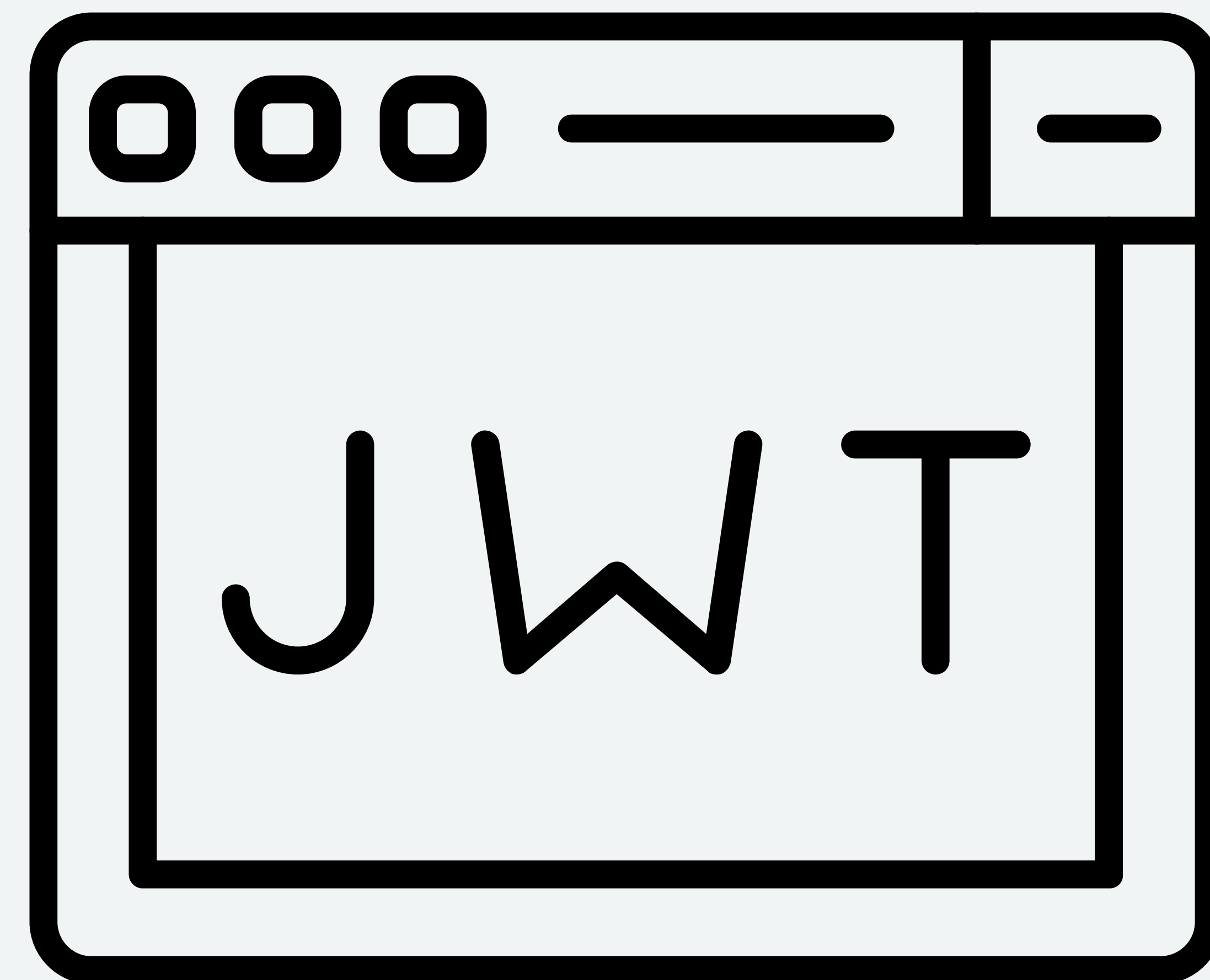
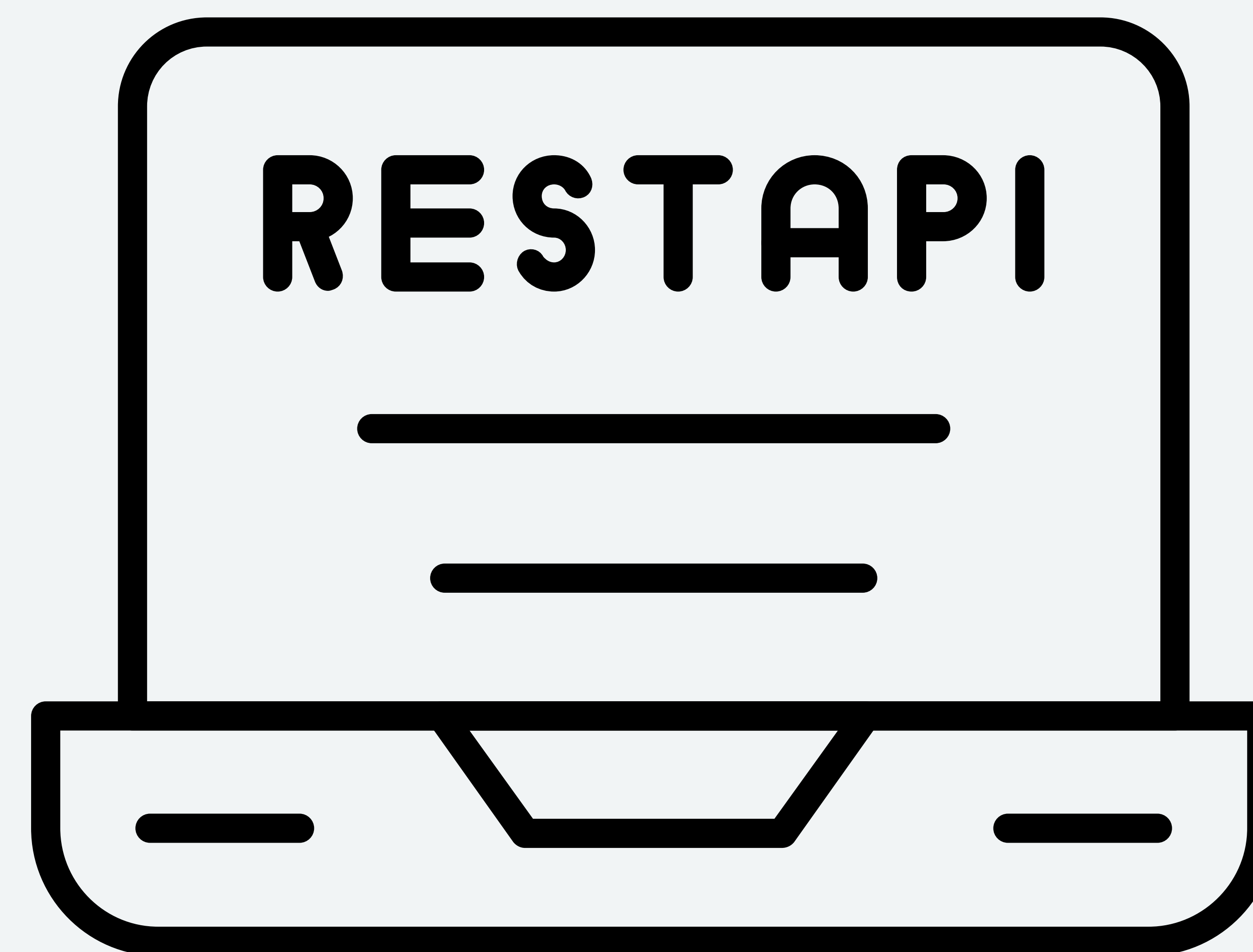
```
http://127.0.0.1:8000/ → Sistema completo
```

```
http://127.0.0.1:8000/recomendacao/ →
```

```
Recomendação pública
```


TECNOLOGIAS:

django



OBJETIVO ACADÊMICO:

Projeto desenvolvido na disciplina de Desenvolvimento Web, com foco em:

- Modelagem de sistemas;
- Arquitetura de software;
- Boas práticas de desenvolvimento;
- Desenvolvimento de aplicações web com API REST.

